Concepts

STRUCTURE	
Self-referential data The data type refers to itself.	*List* HtDP DCIC
	Tree MIT 6.102 ×2
	Nested doll HtDP
Decomposition Problem split into smaller instances.	*Structural* LYAH HtDP ×2 LeetCode
	Generative LYAH HtDP ×2 LeetCode ×4
Base case Where recursion stops.	*Edge condition* LYAH LeetCode ×4
	Pruned leaf LeetCode
Leap of faith LeetCode ×5 The recursive result is trusted, not traced.	
PROCESS	
Return & combine Sub-results flow back and merge.	*Value return* LeetCode ×4
	Result composition SICP HtDP LeetCode ×2
Choice / search Each call picks among options.	*Branching* LeetCode ×3
	Pruning LeetCode
Overlapping subproblems LeetCode ×2 The same subcall recurs.	
Execution state What is held while running.	*Call stack* LeetCode ×2
Cost Work done per call or level.	*Per level* CLRS
	Total CLRS LeetCode

Node Content

Concrete data DCIC MIT 6.102 Values of the input itself.	`[1, 4, 3]`
Call with arguments SICP LeetCode ×3 The call as written, concrete args.	`fib 3`
State tuple LeetCode ×3 Only the changing parameters.	`(i, k)`
Partial expression SICP HtDP LeetCode Half-evaluated term, as in a trace.	`(* 6 (f 5))`
Symbolic CLRS LeetCode Parametric size, as in a recurrence.	*f(n/b)*

Code Correspondence

Diagram is code SICP HtDP DCIC Trace written in the program's own syntax.
Side by side LYAH ×2 SICP LeetCode ×4 Diagram placed next to the code or rule.
Arrows into code HtDP LeetCode Arrows tie parts of the diagram to code.
Shared identifiers SICP LeetCode ×3 Function and parameter names reused in nodes.
Recurrence only CLRS LeetCode ×2 Linked to an equation, not to code.
Prose only LeetCode ×2 Linked through a written explanation.

Visual Encodings

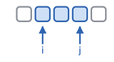
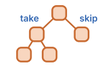
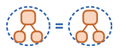
Nested containment Calls drawn inside their caller.	*Box-in-box* HtDP	
	Brackets LYAH LeetCode ×2	`f(a(b(c)))`
Tree (node-link) Calls linked to their subcalls.	*Call tree* SICP LeetCode ×3	
	State tree LeetCode ×2	
	Choice tree LeetCode	
	Result tree LeetCode ×6	
Linear chain List of nodes and links.	*Box-and-pointer* HtDP LeetCode	
	Pointer rewiring LeetCode ×8	
Stack LeetCode Frames piled up by depth.		
Substitution trace SICP HtDP ×3 LeetCode Expression rewritten until a value.		
Indentation staircase DCIC LeetCode Depth shown by indentation.		
Table HtDP LeetCode Args, intermediates and results.		
Partition LeetCode Input split into boxed parts.		
Instance mapping LeetCode ×3 Size n aligned to size n−1.		
Metaphor / pictorial LYAH HtDP LeetCode Dolls, mirrors, spirals.		

Flow Direction

Static HtDP DCIC MIT 6.102 LeetCode ×3 Structure only, no direction.	
Downward only LeetCode ×2 Calls drawn, returns omitted.	
Upward only HtDP Results merging toward the root.	
Round trip SICP LeetCode ×7 Down to the base case, then back up.	

Rewrite in place SICP HtDP ×2
Time runs down the lines.
	`f(3)`
	`⇒ 3·f(2)`
	`⇒ 3·2·f(1)`
	`⇒ 6`

Annotations

Base-case marker LeetCode ×4 Marks where recursion stops.	
Return value LeetCode ×3 Value written on the way back.	
Prune / fail + legend LeetCode ×3 Dead branches, keyed in a legend.	
Role colour LYAH LeetCode ×2 Colour by role (pivot, done).	
Cursor / pointer LeetCode ×3 Indices into the data.	
Choice label LeetCode ×2 The decision named on the edge.	
Call-step label LeetCode ×2 Each frame or step numbered.	
Duplicate marker LeetCode Repeated subproblems circled.	
Traversal path SICP Evaluation order traced.	

Elision

Full expansion SICP LeetCode ×2 Small input, every call drawn.		
Unpatterned elision LeetCode No inference can be made.		
Patterned elision CLRS Rule implied by a few levels.		
Deferred call Subcall left unexpanded.	*Trace term* HtDP	`f(rest…)`
	Done box LeetCode ×2	done

Subdiagrams

Multiple representations HtDP Same recursion, two encodings.
Multiple scenarios LeetCode ×4 Cases of one step side by side.
Over time HtDP LeetCode ×14 Snapshots of one run.

LYAH Learn You a Haskell · SICP Structure and Interpretation of Computer Programs · HtDP How to Design Programs · DCIC A Data-Centric Introduction to Computing · CLRS Introduction to Algorithms
MIT 6.102 MIT 6.102 course notes · LeetCode LeetCode solutions, vote ≥ 5 · ×n number of examples